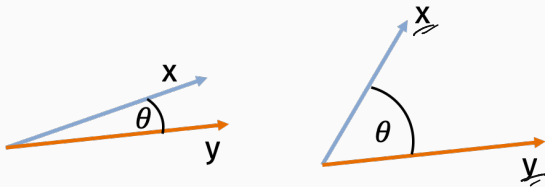


CS-GY 9223 I: Lecture 5

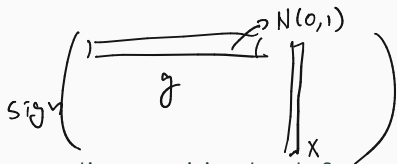
Gradient Descent and Its Many Forms

NYU Tandon School of Engineering, Prof. Christopher Musco

Cosine similarity $\cos(\theta(x, y)) = \frac{\langle x, y \rangle}{\|x\|_2 \|y\|_2}$:



$$-1 \leq \cos(\theta(x, y)) \leq 1.$$



Locality sensitive hash for cosine similarity:

- Let $\mathbf{g} \in \mathbb{R}^d$ be randomly chosen with each entry $\mathcal{N}(0, 1)$.
- h : $\mathbb{R}^d \rightarrow \{-1, 1\}$ is defined $h(\mathbf{x}) = \text{sign}(\langle \mathbf{g}, \mathbf{x} \rangle)$.

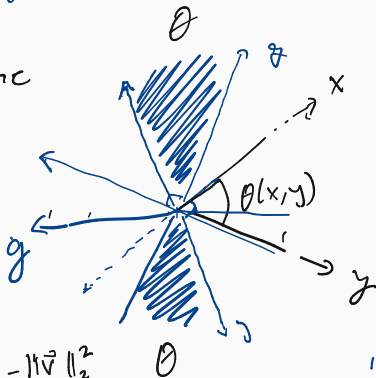
If $\cos(\theta(\mathbf{x}, \mathbf{y})) = \underline{v}$, what is $\Pr[h(\mathbf{x}) == h(\mathbf{y})]$?

SIMHASH ANALYSIS

Dimension: $2 = d$

g = random gaussian

Rotation Invariance



$$\Pr(\vec{g}_i = \vec{v}) \sim \frac{e^{-\|\vec{v}\|_2^2}}{2\pi}$$

$$\Pr(g_i = v) \sim e^{-v^2}$$

$$h(x) = \text{sign}(g^T x) = -1$$

$$h(y) = \text{sign}(g^T y) = 1$$

$$h(x) \neq h(y)$$

$$h(x) = -1$$

$$h(y) = 1$$

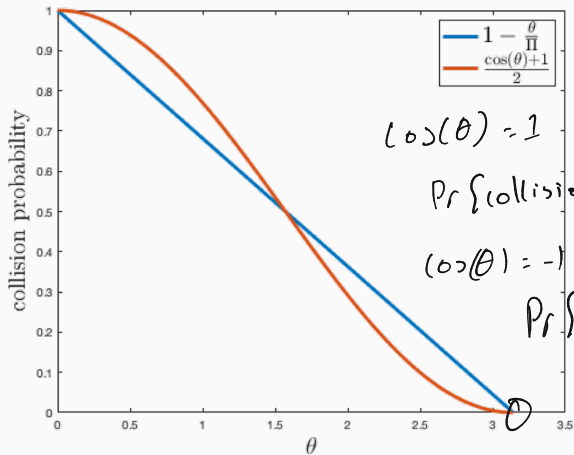
$$h(x) \neq h(y)$$

$$\Pr[h(x) = h(y)]$$

$$\Pr[h(x) \neq h(y)] = \frac{2\theta}{2\pi} = \frac{\theta}{\pi}$$

$$\Pr[h(x) = h(y)] = 1 - \theta/\pi$$

SIMHASH ANALYSIS



$$\cos(\theta) = 1$$

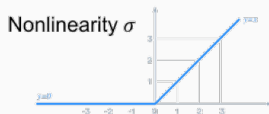
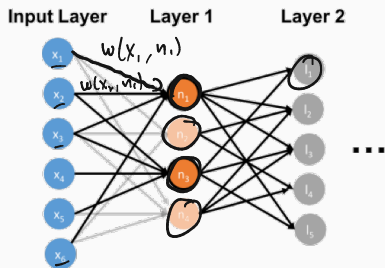
$$\Pr[\text{collision}] = 1$$

$$\cos(\theta) = -1$$

$$\Pr[\text{collision}] = 0$$

SIMHASH TO SPEEDUP NEURAL NETWORKS

Work of Anshumali Shrivastava at Rice University and coauthors.



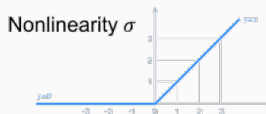
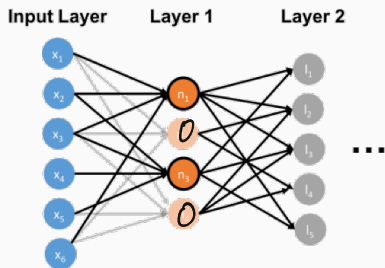
$$n_i = \sigma \left(\sum_{j=1}^m w(x_j, n_i) \cdot x_j \right) = \sigma(\langle \underline{w}_i, \underline{x} \rangle)$$

Handwritten notes show the vector $\underline{w}_i$ as $\begin{bmatrix} w(x_1, n_i) \\ w(x_2, n_i) \\ \vdots \\ w(x_6, n_i) \end{bmatrix}$.

- Number of multiplications to evaluate $\mathcal{N}(\mathbf{x})$: $(|\mathbf{x}| \cdot |\text{layer 1}|) + |\text{layer 1}| \cdot |\text{layer 2}| + |\text{layer 2}| \cdot |\text{layer 3}| + \dots$
- For an approximate solution, only consider neurons on each each with high activation.

SIMHASH TO SPEEDUP NEURAL NETWORKS

Work of Anshumali Shrivastava at Rice University and coauthors.



$$n_i = \sigma \left(\sum_{j=1}^m w(x_j, n_i) \cdot x_j \right) = \sigma(\langle \mathbf{w}_i, \mathbf{x} \rangle)$$

$\mathbf{w}_1$
 $\mathbf{w}_2$
 $\vdots$
 $\mathbf{w}_m$

- High activation = large value of $\sigma(\langle \mathbf{w}_i, \mathbf{x} \rangle)$.
- Typically $\sigma(\langle \mathbf{w}_i, \mathbf{x} \rangle)$ increases as $\langle \mathbf{w}_i, \mathbf{x} \rangle$ increases.
- Use LSH/SimHash to quickly find all $\mathbf{w}_i$ for which $\langle \mathbf{w}_i, \mathbf{x} \rangle$ is large and only include these terms in the sum.

Optimization

Have some function $f: \underline{\mathbb{R}^d} \rightarrow \underline{\mathbb{R}}$. Want to find $\underline{\hat{\mathbf{x}}}$ such that:

$$\underline{f(\hat{\mathbf{x}})} = \min_{\mathbf{x}} f(\mathbf{x}) + \epsilon$$

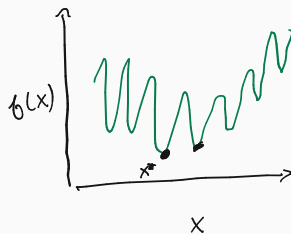
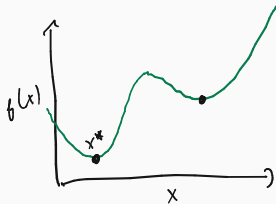
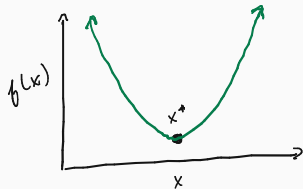
Or at least $\hat{\mathbf{x}}$ is close to a minimum. E.g. $f(\hat{\mathbf{x}}) \leq \min_{\mathbf{x}} f(\mathbf{x}) + \epsilon$

Often we have some additional constraints:

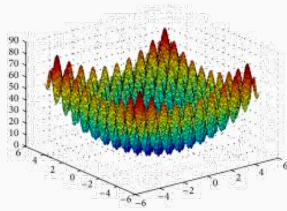
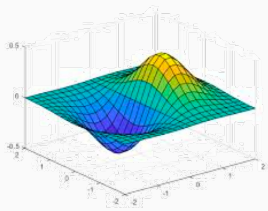
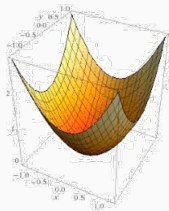
- $\mathbf{x} > 0$
- $\|\mathbf{x}\|_2 \leq R, \|\mathbf{x}\|_1 \leq R$
- $\underline{\mathbf{a}^T \mathbf{x} > c.}$

OPTIMIZATION

Dimension $d = 1$:



Dimension $d = 2$:



Machine learning: Want to learn a model that maps input

- numerical data vectors
- images, video
- text documents

to prediction

- numerical value (probability stock price increases)
- label (is the image a cat? does the image contain a car?)
- decision (turn car left, rotate robotic arm)

Let M_x be a model with parameters $\underline{x = \{x_1, \dots, x_d\}}$.

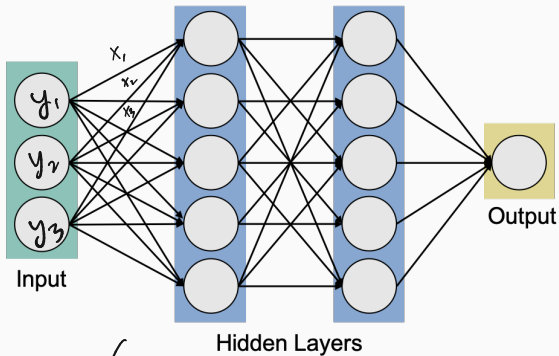
Example:

$$M_x(y) = \text{sign}(y^T \underbrace{x_{1:d-1}}_{\text{data vector}} + \underbrace{x_d}_{\text{param. vector}}) \rightarrow \{-1, 1\}$$

MACHINE LEARNING MODEL

Example:

$X \rightarrow$ weights



M_x

SUPERVISED LEARNING

Classic approach in supervised learning: Find a model that works well on data that you already have the answer for (labels, values, classes, etc.).

- Model $M_{\mathbf{x}}$ parameterized by a vector of numbers $\mathbf{x}$.
- Dataset $\underline{\mathbf{y}}^{(1)}, \dots, \underline{\mathbf{y}}^{(n)}$ with outputs $\underline{o}^{(1)}, \dots, \underline{o}^{(n)}$.

Want to find $\hat{\mathbf{x}}$ so that $M_{\hat{\mathbf{x}}}(\mathbf{y}^{(i)}) \approx o^{(i)}$ for $i \in 1, \dots, n$.

How do we turn this into a function minimization problem?

$$\min f \quad f: \mathbb{R}^d \rightarrow \mathbb{R}$$

LOSS FUNCTION

Loss function $L(M_x(y), o)$: Some measure of distance between prediction $M_x(y)$ and true output o . Increases if they are further apart.

- Squared (ℓ_2) loss: $|M_x(y) - o|^2$
- Absolute deviation (ℓ_1) loss: $|M_x(y) - o|$
- Hinge loss: $1 - o \cdot M_x(y)$ $o, M_x(y) \in \{-1, 1\}$ $\text{Loss} = 0$
 $\text{Loss} = 2$
- Cross-entropy loss (log loss).
- Etc.

Empirical risk minimization:

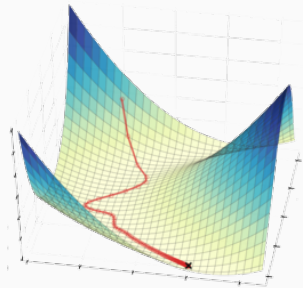
$$f(x) = \sum_{i=1}^n L(\underbrace{M_x(y^{(i)})}_{\text{prediction}}, \underbrace{o^{(i)}}_{\text{target}})$$

Solve the optimization problem $\min_x f(x)$.

Choose $\hat{x}$ s.t. $f(\hat{x}) \leq \min_x f(x) + \varepsilon$

GRADIENT DESCENT

Gradient descent: A greedy algorithm for minimizing functions of multiple variables that often works amazingly well.



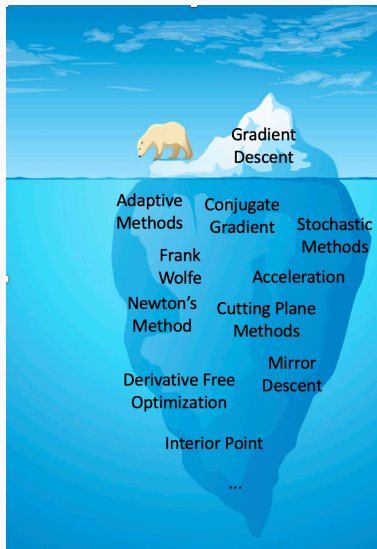
Abraham Maslow:

“I suppose it is tempting, if the only tool you have is a hammer, to treat everything as if it were a nail.”



OPTIMIZATION ALGORITHMS

So much to learn!



CALCULUS REVIEW

For $i = 1, \dots, d$, let x_i be the i^{th} entry of $\mathbf{x}$. Let $\mathbf{e}^{(i)}$ be the i^{th} standard basis vector.

$$\{0 \ 0 \ 0 \ \underset{i}{1} \ 0 \ 0 \ 0\}$$

$$f(\mathbf{x} + t\mathbf{e}^{(i)}) - f(\mathbf{x})$$

Partial derivative:

$$\frac{\partial f}{\partial x_i}(\mathbf{x}) = \lim_{t \rightarrow 0} \frac{f(\mathbf{x} + \underline{t\mathbf{e}^{(i)}}) - f(\mathbf{x})}{t}$$

Directional derivative:

$$D_{\mathbf{v}}f(\mathbf{x}) = \lim_{t \rightarrow 0} \frac{f(\mathbf{x} + \underline{t\mathbf{v}}) - f(\mathbf{x})}{t}$$

$$t \text{ close to } 0 \quad \frac{f(\mathbf{x} + t\mathbf{e}^{(i)}) - f(\mathbf{x})}{t} \approx \frac{df}{dx_i} \cdot t$$

CALCULUS REVIEW

$$x \rightarrow x + tv \quad x \rightarrow x + tv_1 e_1 \rightarrow x + tv_1 e_1 + tv_2 e_2$$

Gradient: $f(x) \rightarrow \underline{f(x+tv)} \rightarrow x + tv_1 e_1 + \dots + tv_d e_d$

$$\underline{\nabla f(x)} = \begin{bmatrix} \frac{\partial f}{\partial x_1}(x) \\ \frac{\partial f}{\partial x_2}(x) \\ \vdots \\ \frac{\partial f}{\partial x_d}(x) \end{bmatrix} \quad f(x) \rightarrow f(x) + tv_1 \frac{\partial f}{\partial x_1}$$

$$f(x) \rightarrow f(x) + tv_1 \frac{\partial f}{\partial x_1} + tv_2 \frac{\partial f}{\partial x_2}$$

Directional derivative:

$$f(x) \rightarrow f(x) + tv_1 \frac{\partial f}{\partial x_1} + \dots + tv_d \frac{\partial f}{\partial x_d}$$

$$\underline{D_v f(x)} = \lim_{t \rightarrow 0} \frac{f(x+tv) - f(x)}{t} = \boxed{\nabla f(x)^T v} + \nabla f(x)^T v$$

$$f(x+tv) - f(x) \approx \underline{t \cdot D_v f(x)}$$

Given a function f to minimize, assume we can:

- **Function oracle:** Evaluate $f(x)$ for any x .
- **Gradient oracle:** Evaluate $\nabla f(x)$ for any x .

EXAMPLE GRADIENT EVALUATION

Linear least-squares regression:

- Given $\underline{\mathbf{y}}^{(1)}, \dots, \underline{\mathbf{y}}^{(n)} \in \mathbb{R}^d$, $\underline{b}^{(1)}, \dots, \underline{b}^{(n)} \in \mathbb{R}$.
- Want to minimize $f(\mathbf{x}) = \sum_{i=1}^n (\underline{\mathbf{x}}^T \underline{\mathbf{y}}^{(i)} - b^{(i)})^2$.

$$\begin{aligned}\nabla f(\mathbf{x}) &= \sum_{i=1}^n \nabla \left(\underline{\mathbf{x}}^T \underline{\mathbf{y}}^{(i)} - b^{(i)} \right)^2 \quad (\text{linearity}) \\ &= 2(\underline{\mathbf{x}}^T \underline{\mathbf{y}}^{(i)} - b^{(i)}) \cdot \nabla (\underline{\mathbf{x}}^T \underline{\mathbf{y}}^{(i)} - b^{(i)}) \\ &= \sum_{i=1}^n \underbrace{2(\underline{\mathbf{x}}^T \underline{\mathbf{y}}^{(i)} - b^{(i)})}_{\text{scalar}} \cdot \underbrace{\underline{\mathbf{y}}^{(i)}}_{\text{vector}} = \boxed{}\end{aligned}$$

EXAMPLE GRADIENT EVALUATION

Matrix view:

$$N \begin{bmatrix} y^{(1)} \\ y^{(2)} \\ \vdots \\ y^{(n)} \end{bmatrix} \begin{bmatrix} x \end{bmatrix} - \begin{bmatrix} b^{(1)} \\ b^{(2)} \\ \vdots \\ b^{(n)} \end{bmatrix} \quad \|\cdot\|_2$$

$Y \qquad \qquad \qquad b$

$$x^T Y^T Y x \quad (x+\Delta)^T Y^T Y (x+\Delta)$$

$$\frac{x^T Y^T Y x + 2x^T Y^T Y \Delta + \Delta^T Y^T Y \Delta}{\Delta}$$

$$f(x) = \|Yx - b\|_2^2$$

$$\nabla (x^T Y^T Y x) = 2Y^T Y x$$

quadratic form

$$f(x) = (Yx - b)^T (Yx - b) = \frac{x^T Y^T Y x - 2x^T Y^T b + b^T b}{\downarrow \quad \downarrow \quad \downarrow}$$

$\quad \quad \quad -2Y^T b \quad \quad \quad 0$

$$\nabla f(x) = 2Y^T Y x - 2Y^T b = 2Y^T (Yx - b)$$

DECENT METHODS

$$x^T M x = \sum_{i,j=1}^n x_i x_j M_{ij} \quad \frac{\partial f}{\partial x_i} \left(\sum_{i,j} x_i x_j M_{ij} \right)$$

Greedy approach: Given a starting point x , make a small adjustment that decreases $f(x)$. In particular, $x \leftarrow x + \eta v$ and $f(x) \leftarrow f(x + \eta v)$.

$$v = -\nabla f(x)$$

$$\frac{d}{dx} x^2 = 2x$$

$$\frac{(x+\Delta)^2 - x^2}{\Delta} = \frac{x^2 + 2x\Delta + \Delta^2 - x^2}{\Delta} = 2x + \Delta$$

What property might I want in v ?

$$\nabla f(x)^T v \rightarrow \text{negative}$$

$$D_v f(x) = \lim_{t \rightarrow 0} \frac{f(x + tv) - f(x)}{t} = \nabla f(x)^T v.$$

$$\underbrace{f(x + tv) - f(x)}_{\text{negative}} \approx + \nabla f(x)^T v$$

$$+ \nabla f(x)^T (-\nabla f(x)) = -\|\nabla f(x)\|_2^2$$

$$v = -\nabla f(x^{(i)})$$

Prototype algorithm:

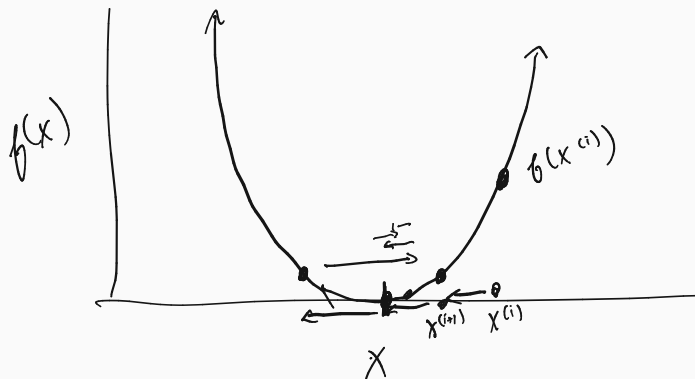
- Choose arbitrary starting point $\underline{x^{(1)}}$.
- For $i = 1, \dots, T$:
 - $\underline{x^{(i+1)}} = \underline{x^{(i)}} - \overset{\text{step size}}{\eta} \nabla f(x^{(i)})$
- Return $x^{(t)}$.

η is a step-size parameter. Needs to be chosen ahead of time or adapted on the go.

$$\min \nabla f(x)^T v \rightarrow -[10000, 0.0001]$$

GRADIENT DESCENT

Example in one dimension:



Why is gradient descent sometimes called
"steepest descent"?

GRADIENT DESCENT

Claim (Gradient descent = Steepest descent)

$$\frac{-\nabla f(x)}{\|\nabla f(x)\|_2} = \arg \min_{\mathbf{v}, \|\mathbf{v}\|_2 \leq 1} \nabla f(x)^T \mathbf{v}$$

Note: We could have chosen to restrict $\mathbf{v}$ using a different norm. What if we had restricted $\|\mathbf{v}\|_1 \leq 1$? $\|\mathbf{v}\|_\infty \leq 1$? These choices lead to variants of generalized steepest descent.

$$\|\mathbf{v}\|_2 \leq 1$$

$$\min_{\mathbf{v}: \|\mathbf{v}\|_2 \leq 1} \nabla f(x)^T \mathbf{v}$$



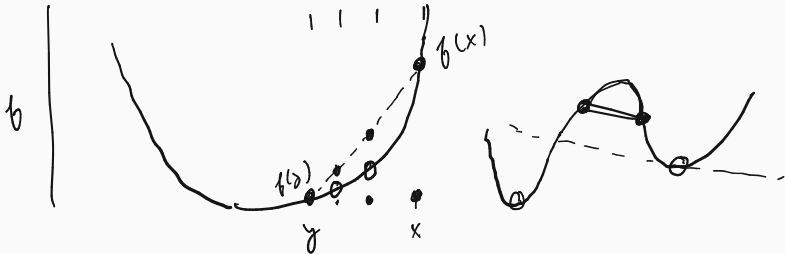
GRADIENT DESCENT

In general, gradient descent can be proven to converge (and we understand how quickly it converges) for convex functions.

Definition (Convex)

A function f is convex iff for any $\underline{x}, \underline{y}, \lambda \in [0, 1]$:

$$(1 - \lambda) \cdot f(x) + \lambda \cdot f(y) \geq f(\underbrace{(1 - \lambda) \cdot x + \lambda \cdot y}_{\text{straight line}})$$



Definition (Convex)

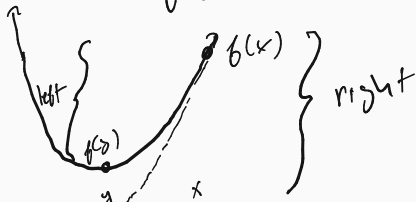
A function f is convex iff for any x, y :

$$\underline{f(x+z) \geq f(x) + \nabla f(x)^T z}$$

$$\underline{f(x) - f(y) \leq \nabla f(x)^T (x - y)}$$

$$z = y - x \quad f(y) \geq f(x) + \nabla f(x)^T (y - x)$$

$$f(y) - f(x) \geq \nabla f(x)^T (y - x)$$



Definition (Convex)

A function f is convex iff for any $\mathbf{x}, \mathbf{y}$:

$$f(\mathbf{x}) - f(\mathbf{y}) \leq \nabla f(\mathbf{x})^T (\mathbf{x} - \mathbf{y})$$



Assume:

- f is convex.
- Lipschitz gradients: for all $\mathbf{x}$, $\|\nabla f(\mathbf{x})\|_2 \leq G$.
- Starting radius: $\|\mathbf{x}^* - \mathbf{x}^{(1)}\|_2 \leq R$.



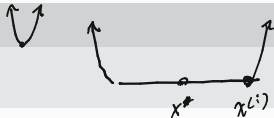
Gradient descent:

- Choose number of steps T .
- $\eta = \frac{R}{G\sqrt{T}}$
- For $i = 1, \dots, T$:
 - $\mathbf{x}^{(i+1)} = \mathbf{x}^{(i)} - \eta \nabla f(\mathbf{x}^{(i)})$
- Return $\hat{\mathbf{x}} = \arg \min_{\mathbf{x}^{(i)}} f(\mathbf{x}^{(i)})$.
- Alternatively, return $\hat{\mathbf{x}} = \frac{1}{T} \sum_{i=1}^T \mathbf{x}^{(i)}$.

GRADIENT DESCENT ANALYSIS

Claim (GD Convergence Bound)

If $T \geq \frac{R^2 G^2}{\epsilon^2}$, then $f(\hat{\mathbf{x}}) \leq f(\mathbf{x}^*) + \epsilon$.



$$\begin{aligned} \underbrace{\|x^{(i+1)} - x^*\|_2^2} &= \|x^{(i)} - \eta \nabla f(x^{(i)}) - x^*\|_2^2 \\ &= \|x^{(i)} - x^*\|_2^2 + \eta^2 \|\nabla f(x^{(i)})\|_2^2 - \underbrace{2\eta \nabla f(x^{(i)}) (x^{(i)} - x^*)}_{\leq G^2} \end{aligned}$$

$$\underbrace{\nabla f(x^{(i)}) (x^{(i)} - x^*)}_{\leq G^2} = \frac{\|x^{(i)} - x^*\|_2^2 - \|x^{(i+1)} - x^*\|_2^2}{2\eta} + \frac{\eta}{2} \underbrace{\|\nabla f(x^{(i)})\|_2^2}_{\leq G^2}$$

$$f(x^{(i)}) - f(x^*) \leq \nabla f(x^{(i)}) (x^{(i)} - x^*)$$

$$f(x^{(i)}) - f(x^*) \leq \frac{\|x^{(i)} - x^*\|_2^2 - \|x^{(i+1)} - x^*\|_2^2}{2\eta} + \frac{\eta}{2} G^2 \quad \text{for all } i$$

GRADIENT DESCENT ANALYSIS

Claim (GD Convergence Bound)

If $T \geq \frac{R^2 G^2}{\epsilon^2}$, then $f(\hat{x}) \leq f(x^*) + \epsilon$.

$$n = \frac{B}{G\sqrt{T}}$$

$$f(x^{(i)}) - f(x^*) \leq \frac{\|x^{(i)} - x^*\|_2^2 - \|x^{(i+1)} - x^*\|_2^2}{2n} + \frac{n}{2} G^2 \quad \text{for all } i$$

$$\begin{aligned} \sum_{i=1}^T f(x^{(i)}) - f(x^*) &\leq \underbrace{\sum_{i=1}^{T-1} \frac{\|x^{(i)} - x^*\|_2^2 - \|x^{(i+1)} - x^*\|_2^2}{2n}}_{2n} + \frac{Tn}{2} G^2 \\ &= \frac{\|x^{(1)} - x^*\|_2^2 - \|x^{(T)} - x^*\|_2^2}{2n} - \frac{Tn}{2} G^2 \end{aligned}$$

$$\leq \frac{B^2}{2n} + \frac{Tn}{2} G^2 \leq \frac{B G}{2\sqrt{T}} + \frac{B G}{2\sqrt{T}}$$

$$\sum_{i=1}^T f(x^{(i)}) - f(x^*) \leq B G \sqrt{T}$$

GRADIENT DESCENT ANALYSIS

Claim (GD Convergence Bound)

If $T \geq \frac{R^2 G^2}{\epsilon^2}$, then $f(\hat{x}) \leq f(x^*) + \epsilon$. $\frac{1}{\epsilon} \log(1/\epsilon)$

$$\sum_{i=1}^T f(x^{(i)}) - f(x^*) \leq RG\sqrt{T}$$

$$\frac{1}{T} \left[\sum_{i=1}^T f(x^{(i)}) - f(x^*) \right] \leq \frac{RG}{\sqrt{T}} \leq \epsilon \quad \text{when } T \geq \frac{R^2 G^2}{\epsilon^2}$$

$$f\left(\frac{1}{T} \sum_{i=1}^T x^{(i)}\right) - f(x^*) \leq \frac{1}{T} \left[\sum_{i=1}^T f(x^{(i)}) - f(x^*) \right]$$

or $\hat{x} \rightarrow \left[f(\hat{x}) - f(x^*) \leq \epsilon \right]$

$$\min_i f(x^{(i)}) - f(x^*) \leq \frac{1}{T} \sum_{i=1}^T f(x^{(i)}) - f(x^*)$$

ONLINE GRADIENT DESCENT

Instead of a single function f to minimize, assume we have an unknown and changing set of objective functions:

$$f_1, \dots, f_T.$$

- At each time step, choose $\mathbf{x}^{(i)}$.
- f_i is revealed and we pay cost $f_i(\mathbf{x}^{(i)})$
- **Goal:** Minimize $\underbrace{\sum_{i=1}^T f_i(\mathbf{x}^{(i)})}$.

$$\|xw - b\|$$

$$x^{(1)} \dots x^{(T)}$$

$$\|Ax - b\|^2$$

REGRET BOUND

Objective: Choose $\underline{x^{(1)}}, \dots, \underline{x^{(T)}}$ so that:

$$\underbrace{\sum_{i=1}^T f_i(x^{(i)})}_{\text{}} \leq \left[\min_x \sum_{i=1}^T f_i(x) \right] + \underbrace{\Delta}_{\text{"regret"}}$$

We want to compete with the best fixed solution in hindsight.

$$\text{opt} = x^*$$

Assume:

- Lipschitz gradients: for all $\mathbf{x}, i$, $\|\nabla f_i(\mathbf{x})\|_2 \leq G$.
- Starting radius: $\|\mathbf{x}^* - \mathbf{x}^{(1)}\|_2 \leq R$.

Online Gradient descent:

- Choose number of steps T .
- $\eta = \frac{\beta}{G\sqrt{T}}$
- For $i = 1, \dots, T$:
 - $\mathbf{x}^{(i+1)} = \mathbf{x}^{(i)} - \eta \nabla f_i(\mathbf{x}^{(i)})$
- Play $\mathbf{x}^{(i+1)}$.

Claim (OGD Regret Bound)

After T steps, $\Delta = \left[\sum_{i=1}^T f_i(x^{(i)}) \right] - \left[\sum_{i=1}^T f_i(x^*) \right] \leq RG\sqrt{T}$

$$\nabla f_i(x^{(i)}) (x^{(i)} - x^*) \leq \frac{\|x^{(i)} - x^*\|_2^2 - \|x^{(i+1)} - x^*\|_2^2}{2\eta} + \frac{\eta G^2}{2}$$

$$f_i(x^{(i)}) - f_i(x^*) \leq$$

$$\sum_{i=1}^T f_i(x^{(i)}) - f_i(x^*) \leq \frac{\|x^{(1)} - x^*\|_2^2 - \|x^{(T)} - x^*\|_2^2}{2\eta} + \frac{\eta G^2}{2}$$

$$\leq \frac{R^2}{2\eta} + \frac{\eta G^2}{2} = \underline{\underline{RG\sqrt{T}}}$$

$$\frac{1}{T} \sum_{i=1}^T f_i(x^{(i)}) - f_i(x^*) \leq \frac{RG}{\sqrt{T}}$$

Claim (OGD Regret Bound)

After T steps, $\Delta = \left[\sum_{i=1}^T f_i(\mathbf{x}^{(i)}) \right] - \left[\sum_{i=1}^T f_i(\mathbf{x}^*) \right] \leq RG\sqrt{T}$

Recall the machine learning setup. In empirical risk minimization, we can typically write:

$$f(\mathbf{x}) = \sum_{j=1}^n f_j(\mathbf{x})$$

where f_j is the loss function for a particular data point.

Linear regression:

$$f(\mathbf{x}) = \sum_{j=1}^n (\mathbf{x}^T \mathbf{y}^{(j)} - b^{(j)})^2$$

Pick random $j \in 1, \dots, n$:

$$\mathbb{E} [\nabla f_j(\mathbf{x})] = \nabla f(\mathbf{x}).$$

But $\nabla f_j(\mathbf{x})$ can be computed in a $1/n$ fraction of the time!

Main idea: Use random approximate gradient in place of actual gradient.

Trade slower convergence for cheaper iterations.

Assume:

- Lipschitz gradients: for all $\mathbf{x}, j$, $\|\nabla f_j(\mathbf{x})\|_2 \leq \frac{G'}{n}$.
- Starting radius: $\|\mathbf{x}^* - \mathbf{x}^{(1)}\|_2 \leq R$.

Stochastic Gradient descent:

- Choose number of steps T .
- $\eta = \frac{D}{G'\sqrt{T}}$
- For $i = 1, \dots, T$:
 - Pick random $j_i \in 1, \dots, n$.
 - $\mathbf{x}^{(i+1)} = \mathbf{x}^{(i)} - \eta \nabla f_{j_i}(\mathbf{x}^{(i)})$
- Return $\hat{\mathbf{x}} = \frac{1}{T} \sum_{i=1}^T \mathbf{x}^{(i)}$

Claim (SGD Convergence)

After $T = \frac{R^2 G^2}{\epsilon^2}$ iteration:

$$\mathbb{E} [f(\hat{\mathbf{x}}) - f(\mathbf{x}^*)] \leq \epsilon.$$

Claim (SGD Convergence)

After $T = \frac{R^2 G^2}{\epsilon^2}$ iteration:

$$\mathbb{E} [f(\hat{\mathbf{x}}) - f(\mathbf{x}^*)] \leq \epsilon.$$

Claim (SGD Convergence)

After $T = \frac{R^2 G^2}{\epsilon^2}$ iteration:

$$\mathbb{E} [f(\hat{\mathbf{x}}) - f(\mathbf{x}^*)] \leq \epsilon.$$

COMPARISON

Number of iterations for error ϵ :

- Gradient Descent: $T = \frac{R^2 G^2}{\epsilon^2}$.
- Stochastic Gradient Descent: $T = \frac{R^2 G'^2}{\epsilon^2}$.

Always have $G \leq G'$:

$$\|\nabla f(x)\|_2 \leq \|\nabla f_1(x)\|_2 + \dots + \|\nabla f_n(x)\|_2 \leq n \cdot \frac{G'}{n} = G'.$$

Fair comparison:

$$\frac{R^2 G'^2}{\epsilon^2} = n \cdot \frac{R^2 G^2}{\epsilon^2}$$